

```
from google.colab import drive
drive.mount('/content/drive')
```

Mounted at /content/drive

```
import zipfile
```

```
import zipfile
import os

zip_path = "/content/drive/MyDrive/task-viii-files.zip"
extract_path = "/content/dataset"

with zipfile.ZipFile(zip_path, 'r') as zip_ref:
    zip_ref.extractall(extract_path)

print("Dataset extracted")

for root, dirs, files in os.walk("/content/dataset"):
    print(root, "->", len(files))
```

```
Dataset extracted
/content/dataset -> 0
/content/dataset/__MACOSX -> 1
/content/dataset/__MACOSX/Samples -> 10000
/content/dataset/Samples -> 10000
```

```
import os

sample_path = "/content/dataset/Samples"

files = os.listdir(sample_path)
print("Total files:", len(files))

print(files[:10])
```

```
Total files: 10000
['sample730.npy', 'sample3176.npy', 'sample2285.npy', 'sample9833.npy', 'sample2021.npy', 'sample5531.npy', 'sample5257.npy']
```

```
import os
import numpy as np
import matplotlib.pyplot as plt
import random

sample_path = "/content/dataset/Samples"
files = os.listdir(sample_path)

print("Total files:", len(files))
print(files[:10])

plt.figure(figsize=(8,8))

for i in range(9):
    f = random.choice(files)
    img = np.load(os.path.join(sample_path, f))

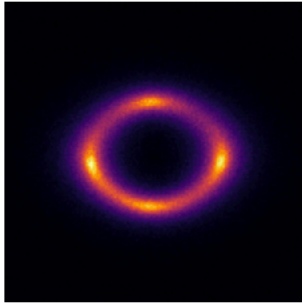
    plt.subplot(3,3,i+1)
    plt.imshow(img.transpose(1,2,0), cmap="inferno")
    plt.title(f)
    plt.axis("off")

plt.tight_layout()
plt.show()
```

Total files: 10000

['sample730.npy', 'sample3176.npy', 'sample2285.npy', 'sample9833.npy', 'sample2021.npy', 'sample5531.npy', 'sample5257.npy']

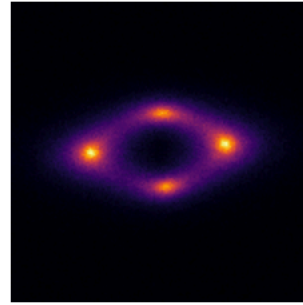
sample6871.npy



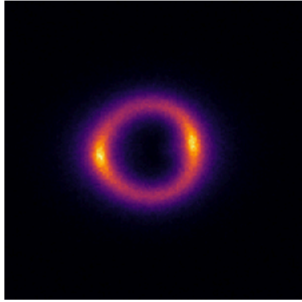
sample2784.npy



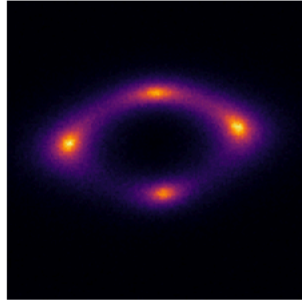
sample7101.npy



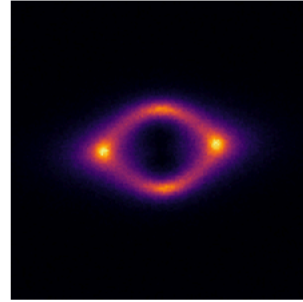
sample3362.npy



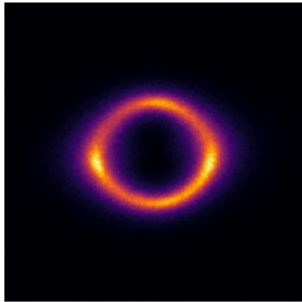
sample4653.npy



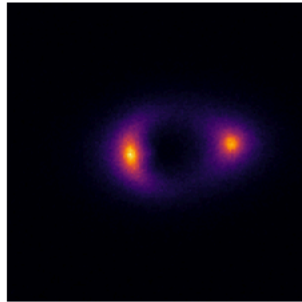
sample155.npy



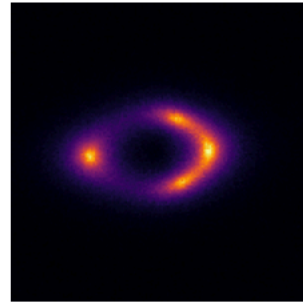
sample2376.npy



sample7880.npy



sample5518.npy



```
import os
import math
import random
import numpy as np
import torch
import torch.nn as nn
import torch.nn.functional as F
from torch.utils.data import Dataset, DataLoader, random_split
from torchvision.utils import save_image
from tqdm import tqdm
import shutil
```

```
class LensingDataset(Dataset):
    def __init__(self, root_dir, image_size=64):
        self.root = root_dir
        self.files = sorted([f for f in os.listdir(root_dir) if f.endswith('.npy')])
        self.size = image_size

        xs = torch.linspace(-1, 1, image_size)
        grid_x, grid_y = torch.meshgrid(xs, xs, indexing='ij')
        self.coord = torch.stack([grid_x, grid_y]) # 2 channels
        self.radial = torch.sqrt(grid_x**2 + grid_y**2).unsqueeze(0) # 1 channel

    def __len__(self):
        return len(self.files)

    def normalize(self, img):
        img = torch.log1p(img)
        img = (img - img.min()) / (img.max() - img.min() + 1e-8)
        return img * 2 - 1

    def __getitem__(self, idx):
        path = os.path.join(self.root, self.files[idx])
        img = np.load(path)
        img = torch.from_numpy(img).float()
        if img.ndim == 3: img = img.mean(0)
```

```

img = img.unsqueeze(0).unsqueeze(0)
img = F.interpolate(img, size=(self.size, self.size), mode="bilinear", align_corners=False).squeeze()

img = self.normalize(img)

return torch.cat([img.unsqueeze(0), self.radial, self.coord], dim=0)

```

```

class SinusoidalPosEmb(nn.Module):
    def __init__(self, dim):
        super().__init__()
        self.dim = dim

    def forward(self, x):
        device = x.device
        half_dim = self.dim // 2
        emb = math.log(10000) / (half_dim - 1)
        emb = torch.exp(torch.arange(half_dim, device=device) * -emb)
        emb = x[:, None] * emb[None, :]
        emb = torch.cat((emb.sin(), emb.cos()), dim=-1)
        return emb

class ResBlock(nn.Module):
    def __init__(self, in_ch, out_ch, time_dim):
        super().__init__()
        self.time_mlp = nn.Linear(time_dim, out_ch)
        self.conv1 = nn.Sequential(
            nn.GroupNorm(32, in_ch), nn.SiLU(), nn.Conv2d(in_ch, out_ch, 3, padding=1)
        )
        self.conv2 = nn.Sequential(
            nn.GroupNorm(32, out_ch), nn.SiLU(), nn.Conv2d(out_ch, out_ch, 3, padding=1)
        )
        self.res_conv = nn.Conv2d(in_ch, out_ch, 1) if in_ch != out_ch else nn.Identity()

    def forward(self, x, t):
        h = self.conv1(x)
        h += self.time_mlp(t)[:, :, None, None]
        h = self.conv2(h)
        return h + self.res_conv(x)

class Attention(nn.Module):
    def __init__(self, dim):
        super().__init__()
        self.norm = nn.GroupNorm(32, dim)
        self.qkv = nn.Conv2d(dim, dim * 3, 1, bias=False)
        self.proj = nn.Conv2d(dim, dim, 1)

    def forward(self, x):
        b, c, h, w = x.shape
        q, k, v = self.qkv(self.norm(x)).reshape(b, 3, c, h*w).unbind(1)
        attn = (q.transpose(-1, -2) @ k) * (c ** -0.5)
        attn = attn.softmax(dim=-1)
        out = (v @ attn.transpose(-1, -2)).reshape(b, c, h, w)
        return self.proj(out) + x

```

```

class DiffusionUNet(nn.Module):
    def __init__(self, base_ch=64):
        super().__init__()
        self.time_mlp = nn.Sequential(
            SinusoidalPosEmb(base_ch),
            nn.Linear(base_ch, base_ch * 4),
            nn.SiLU(),
            nn.Linear(base_ch * 4, base_ch * 4)
        )
        t_dim = base_ch * 4

        self.init_conv = nn.Conv2d(4, base_ch, 3, padding=1)

        self.d1 = ResBlock(base_ch, base_ch, t_dim)
        self.down_conv2 = nn.Conv2d(base_ch, base_ch*2, 4, 2, 1)
        self.d2_resblock = ResBlock(base_ch*2, base_ch*2, t_dim)

        self.down_conv3 = nn.Conv2d(base_ch*2, base_ch*4, 4, 2, 1)
        self.d3_resblock = ResBlock(base_ch*4, base_ch*4, t_dim)

        self.mid1 = ResBlock(base_ch*4, base_ch*4, t_dim)
        self.mid_attn = Attention(base_ch*4)
        self.mid2 = ResBlock(base_ch*4, base_ch*4, t_dim)

        self.u3 = nn.ConvTranspose2d(base_ch*4, base_ch*2, 4, 2, 1)
        self.u3_res = ResBlock(base_ch*4, base_ch*2, t_dim)

```

```

self.u2 = nn.ConvTranspose2d(base_ch*2, base_ch, 4, 2, 1)
self.u2_res = ResBlock(base_ch*2, base_ch, t_dim)

self.final = nn.Sequential(
    nn.GroupNorm(32, base_ch), nn.SiLU(), nn.Conv2d(base_ch, 1, 3, padding=1)
)

def forward(self, x, t):
    t = self.time_mlp(t)
    h = self.init_conv(x)

    s1 = self.d1(h, t)

    h = self.down_conv2(s1)
    s2 = self.d2_resblock(h, t)

    h = self.down_conv3(s2)
    h = self.d3_resblock(h, t)

    h = self.mid1(h, t)
    h = self.mid_attn(h)
    h = self.mid2(h, t)

    h = self.u3(h)
    h = self.u3_res(torch.cat([h, s2], dim=1), t)
    h = self.u2(h)
    h = self.u2_res(torch.cat([h, s1], dim=1), t)
    return self.final(h)

class EMA:
    def __init__(self, beta=0.999):
        self.beta = beta
    def step(self, model, ema_model):
        for p, ema_p in zip(model.parameters(), ema_model.parameters()):
            ema_p.data.mul_(self.beta).add_(p.data, alpha=1 - self.beta)

```

```

from google.colab import drive
drive.mount('/content/drive')

```

Mounted at /content/drive

```
import zipfile
```

```

import zipfile
import os

zip_path = "/content/drive/MyDrive/task-viii-files.zip"
extract_path = "/content/dataset"

with zipfile.ZipFile(zip_path, 'r') as zip_ref:
    zip_ref.extractall(extract_path)

print("Dataset extracted")

for root, dirs, files in os.walk("/content/dataset"):
    print(root, "->", len(files))

```

```

Dataset extracted
/content/dataset -> 0
/content/dataset/Samples -> 10000
/content/dataset/__MACOSX -> 1
/content/dataset/__MACOSX/Samples -> 10000

```

```

device = "cuda"
T = 1000
betas = torch.linspace(1e-4, 0.02, T).to(device)
alphas = 1. - betas
alphas_cumprod = torch.cumprod(alphas, dim=0)

model = DiffusionUNet(base_ch=64).to(device)
ema_model = DiffusionUNet(base_ch=64).to(device)
ema_model.load_state_dict(model.state_dict())
ema_logic = EMA()
optimizer = torch.optim.AdamW(model.parameters(), lr=2e-4)

full_dataset = LensingDataset("/content/dataset/Samples")

train_size = int(0.90 * len(full_dataset))
val_size = len(full_dataset) - train_size

```

```
train_ds, val_ds = random_split(full_dataset, [train_size, val_size])
```

```
loader = DataLoader(  
    train_ds,  
    batch_size=32,  
    shuffle=True,  
    num_workers=4  
)
```

```
val_loader = DataLoader(  
    val_ds,  
    batch_size=32,  
    shuffle=False,  
    num_workers=4  
)
```

```
print("Train size:", len(train_ds))  
print("Val size:", len(val_ds))
```

```
Train size: 9000  
Val size: 1000  
/usr/local/lib/python3.12/dist-packages/torch/utils/data/dataloader.py:424: UserWarning: This DataLoader will create 4 worker  
    self.check_worker_number_rationality()
```

```
train_size
```

```
9000
```

```
len(train_ds)
```

```
9000
```

```
len(val_ds)
```

```
1000
```

```
best_val_loss = float("inf")
```

```
for epoch in range(50):
```

```
    model.train()  
    pbar = tqdm(loader)
```

```
    for batch in pbar:
```

```
        optimizer.zero_grad()
```

```
        img = batch.to(device)
```

```
        t = torch.randint(0, T, (img.shape[0],), device=device)
```

```
        noise = torch.randn_like(img[:,0:1])
```

```
        alpha_hat = alphas_cumprod[t][:,None,None,None]
```

```
        xt = torch.sqrt(alpha_hat) * img[:,0:1] + torch.sqrt(1-alpha_hat) * noise
```

```
        pred_noise = model(torch.cat([xt, img[:,1:]], dim=1), t)
```

```
        loss = F.mse_loss(pred_noise, noise)
```

```
        loss.backward()
```

```
        optimizer.step()
```

```
        ema_logic.step(model, ema_model)
```

```
        pbar.set_description(f"Epoch {epoch} Train Loss: {loss.item():.4f}")
```

```
    model.eval()
```

```
    val_loss = 0
```

```
    with torch.no_grad():
```

```
        for batch in val_loader:
```

```
            img = batch.to(device)
```

```

t = torch.randint(0, T, (img.shape[0],), device=device)

noise = torch.randn_like(img[:,0:1])

alpha_hat = alphas_cumprod[t][:,None,None,None]

xt = torch.sqrt(alpha_hat) * img[:,0:1] + torch.sqrt(1-alpha_hat) * noise

pred_noise = model(torch.cat([xt, img[:,1:]], dim=1), t)

loss = F.mse_loss(pred_noise, noise)

val_loss += loss.item()

val_loss /= len(val_loader)

print(f"Epoch {epoch} | Val Loss: {val_loss:.5f}")

if val_loss < best_val_loss:
    best_val_loss = val_loss
    best_weights = ema_model.state_dict()

print("New best model")

```

```

0%|          | 0/282 [00:00<?, ?it/s]/usr/local/lib/python3.12/dist-packages/torch/utils/data/dataloader.py:432: UserWar
self.check_worker_number_rationality()
Epoch 0 Train Loss: 0.0109: 100%|██████████| 282/282 [00:46<00:00, 6.03it/s]
Epoch 0 | Val Loss: 0.01443
New best model
Epoch 1 Train Loss: 0.0032: 100%|██████████| 282/282 [00:46<00:00, 6.10it/s]
Epoch 1 | Val Loss: 0.01054
New best model
Epoch 2 Train Loss: 0.0035: 100%|██████████| 282/282 [00:45<00:00, 6.14it/s]
Epoch 2 | Val Loss: 0.00720
New best model
Epoch 3 Train Loss: 0.0034: 100%|██████████| 282/282 [00:46<00:00, 6.12it/s]
Epoch 3 | Val Loss: 0.00664
New best model
Epoch 4 Train Loss: 0.0037: 100%|██████████| 282/282 [00:45<00:00, 6.14it/s]
Epoch 4 | Val Loss: 0.00796
Epoch 5 Train Loss: 0.0033: 100%|██████████| 282/282 [00:46<00:00, 6.12it/s]
Epoch 5 | Val Loss: 0.00590
New best model
Epoch 6 Train Loss: 0.0072: 100%|██████████| 282/282 [00:45<00:00, 6.14it/s]
Epoch 6 | Val Loss: 0.00692
Epoch 7 Train Loss: 0.0016: 100%|██████████| 282/282 [00:45<00:00, 6.14it/s]
Epoch 7 | Val Loss: 0.00576
New best model
Epoch 8 Train Loss: 0.0076: 100%|██████████| 282/282 [00:45<00:00, 6.14it/s]
Epoch 8 | Val Loss: 0.00461
New best model
Epoch 9 Train Loss: 0.0020: 100%|██████████| 282/282 [00:45<00:00, 6.15it/s]
Epoch 9 | Val Loss: 0.00673
Epoch 10 Train Loss: 0.0012: 100%|██████████| 282/282 [00:45<00:00, 6.14it/s]
Epoch 10 | Val Loss: 0.00645
Epoch 11 Train Loss: 0.0066: 100%|██████████| 282/282 [00:45<00:00, 6.14it/s]
Epoch 11 | Val Loss: 0.00579
Epoch 12 Train Loss: 0.0062: 100%|██████████| 282/282 [00:45<00:00, 6.15it/s]
Epoch 12 | Val Loss: 0.00548
Epoch 13 Train Loss: 0.0016: 100%|██████████| 282/282 [00:46<00:00, 6.11it/s]
Epoch 13 | Val Loss: 0.00644
Epoch 14 Train Loss: 0.0010: 100%|██████████| 282/282 [00:45<00:00, 6.14it/s]
Epoch 14 | Val Loss: 0.00617
Epoch 15 Train Loss: 0.0030: 100%|██████████| 282/282 [00:45<00:00, 6.14it/s]
Epoch 15 | Val Loss: 0.00743
Epoch 16 Train Loss: 0.0026: 100%|██████████| 282/282 [00:46<00:00, 6.13it/s]
Epoch 16 | Val Loss: 0.00570
Epoch 17 Train Loss: 0.0008: 100%|██████████| 282/282 [00:45<00:00, 6.14it/s]
Epoch 17 | Val Loss: 0.00423
New best model
Epoch 18 Train Loss: 0.0034: 100%|██████████| 282/282 [00:46<00:00, 6.11it/s]
Epoch 18 | Val Loss: 0.00409
New best model
Epoch 19 Train Loss: 0.0201: 100%|██████████| 282/282 [00:46<00:00, 6.13it/s]
Epoch 19 | Val Loss: 0.00592
Epoch 20 Train Loss: 0.0037: 100%|██████████| 282/282 [00:45<00:00, 6.14it/s]
Epoch 20 | Val Loss: 0.00522
Epoch 21 Train Loss: 0.0026: 100%|██████████| 282/282 [00:46<00:00, 6.13it/s]
Epoch 21 | Val Loss: 0.00571
Epoch 22 Train Loss: 0.0012: 100%|██████████| 282/282 [00:46<00:00, 6.13it/s]
Epoch 22 | Val Loss: 0.00522
Epoch 23 Train Loss: 0.0020: 100%|██████████| 282/282 [00:45<00:00, 6.13it/s]
Epoch 23 | Val Loss: 0.00423
New best model

```

```

ema_model.load_state_dict(best_weights)
ema_model.eval()

print("Loaded best EMA model for sampling")

```

```

@torch.no_grad()
def generate_for_fid(model, dataset, n_samples=1000):

    model.eval()

    gen_path = "/content/gen_fid"

    if os.path.exists(gen_path):
        shutil.rmtree(gen_path)

    os.makedirs(gen_path)

    coord = dataset.coord.unsqueeze(0).to(device)
    radial = dataset.radial.unsqueeze(0).to(device)

    for i in tqdm(range(n_samples), desc="Sampling"):

        x = torch.randn((1,1,64,64)).to(device)

        for t_step in reversed(range(T)):

            t = torch.tensor([t_step], device=device)

            p_noise = model(torch.cat([x, radial, coord], dim=1), t)

            a = alphas[t_step]
            a_hat = alphas_cumprod[t_step]

            if t_step > 0:
                fac = (1-a) / torch.sqrt(1-a_hat)

                x = (1/torch.sqrt(a)) * (x - fac * p_noise) \
                    + torch.sqrt(betas[t_step]) * torch.randn_like(x)

            else:
                x = (1/torch.sqrt(a)) * (x - ((1-a)/torch.sqrt(1-a_hat)) * p_noise)

            img_final = (x.clamp(-1,1) + 1) / 2

            save_image(img_final, f"{gen_path}/{i}.png")

generate_for_fid(ema_model, full_dataset)

```

```

Loaded best EMA model for sampling
Sampling: 100%|██████████| 1000/1000 [1:18:38<00:00, 4.72s/it]

```

```

import numpy as np
from torchvision.utils import save_image
import os

real_fid_path = "/content/real_fid"
if os.path.exists(real_fid_path):
    shutil.rmtree(real_fid_path)
os.makedirs(real_fid_path)

all_files = [f for f in os.listdir("/content/dataset/Samples") if f.endswith('.npz')]
selected_files = random.sample(all_files, 1000)

print("Preparing real images for FID comparison...")
for i, fname in enumerate(tqdm(selected_files)):
    img = np.load(os.path.join("/content/dataset/Samples", fname))
    img = torch.from_numpy(img).float()
    if img.ndim == 3: img = img.mean(0)

    img = img.unsqueeze(0).unsqueeze(0)
    img = F.interpolate(img, size=(64, 64), mode="bilinear", align_corners=False).squeeze()

    img = torch.log1p(img)
    img = (img - img.min()) / (img.max() - img.min() + 1e-8)

    save_image(img, f"{real_fid_path}/real_{i}.png")

```

```

Preparing real images for FID comparison...
100%|██████████| 1000/1000 [00:02<00:00, 497.80it/s]

```

```
!pip install pytorch-fid -q
```

```
!python -m pytorch_fid "/content/real_fid" "/content/gen_fid" --device cuda
```

Downloading: "https://github.com/mseitzer/pytorch-fid/releases/download/fid_weights/pt_inception-2015-12-05-6726825d.pth" to
100% 91.2M/91.2M [00:00<00:00, 196MB/s]
100% 20/20 [00:04<00:00, 4.53it/s]
100% 20/20 [00:04<00:00, 4.85it/s]
FID: 2.33970392349309

```
import imageio
import matplotlib.pyplot as plt
from tqdm import tqdm
import os
import torch

gif_dir = "/content/diffusion_gifs"

if os.path.exists(gif_dir):
    shutil.rmtree(gif_dir)

os.makedirs(gif_dir)

@torch.no_grad()
def generate_gif_samples(model, dataset, n_samples=25, steps_to_save=60):

    model.eval()

    coord = dataset.coord.unsqueeze(0).to(device)
    radial = dataset.radial.unsqueeze(0).to(device)

    gif_paths = []

    for i in tqdm(range(n_samples), desc="Generating GIF samples"):

        x = torch.randn((1,1,64,64)).to(device)

        frames = []

        step_interval = T // steps_to_save

        for t_step in reversed(range(T)):

            t = torch.tensor([t_step]).to(device)

            p_noise = model(torch.cat([x, radial, coord], dim=1), t)

            a = alphas[t_step]
            a_hat = alphas_cumprod[t_step]

            if t_step > 0:
                noise = torch.randn_like(x)
            else:
                noise = 0

            x = (1/torch.sqrt(a)) * (x - ((1-a)/torch.sqrt(1-a_hat))*p_noise) + torch.sqrt(betas[t_step])*noise

            if t_step % step_interval == 0 or t_step == 0:

                img = (x.clamp(-1,1)+1)/2
                img = img[0,0].cpu().numpy()

                frames.append((img*255).astype("uint8"))

        gif_path = f"{gif_dir}/sample_{i}.gif"

        imageio.mimsave(
            gif_path,
            frames,
            duration=0.08
        )

        gif_paths.append(gif_path)

    return gif_paths

gif_paths = generate_gif_samples(ema_model, full_dataset)
```

Generating GIF samples: 100%|██████████| 25/25 [02:02<00:00, 4.91s/it]

gif_paths

```

['/content/diffusion_gifs/sample_0.gif',
'/content/diffusion_gifs/sample_1.gif',
'/content/diffusion_gifs/sample_2.gif',
'/content/diffusion_gifs/sample_3.gif',
'/content/diffusion_gifs/sample_4.gif',
'/content/diffusion_gifs/sample_5.gif',
'/content/diffusion_gifs/sample_6.gif',
'/content/diffusion_gifs/sample_7.gif',
'/content/diffusion_gifs/sample_8.gif',
'/content/diffusion_gifs/sample_9.gif',
'/content/diffusion_gifs/sample_10.gif',
'/content/diffusion_gifs/sample_11.gif',
'/content/diffusion_gifs/sample_12.gif',
'/content/diffusion_gifs/sample_13.gif',
'/content/diffusion_gifs/sample_14.gif',
'/content/diffusion_gifs/sample_15.gif',
'/content/diffusion_gifs/sample_16.gif',
'/content/diffusion_gifs/sample_17.gif',
'/content/diffusion_gifs/sample_18.gif',
'/content/diffusion_gifs/sample_19.gif',
'/content/diffusion_gifs/sample_20.gif',
'/content/diffusion_gifs/sample_21.gif',
'/content/diffusion_gifs/sample_22.gif',
'/content/diffusion_gifs/sample_23.gif',
'/content/diffusion_gifs/sample_24.gif']

```

```

import imageio
import matplotlib.pyplot as plt
import torch
import os
import shutil

gif_dir = "/content/diffusion_single"

if os.path.exists(gif_dir):
    shutil.rmtree(gif_dir)

os.makedirs(gif_dir)

@torch.no_grad()
def generate_single_diffusion_gif(model, dataset, steps_to_save=120):

    model.eval()

    coord = dataset.coord.unsqueeze(0).to(device)
    radial = dataset.radial.unsqueeze(0).to(device)

    x = torch.randn((1,1,64,64)).to(device)

    frames = []

    step_interval = max(1, T // steps_to_save)

    print("Generating diffusion trajectory...")

    for t_step in reversed(range(T)):

        t = torch.tensor([t_step]).to(device)

        p_noise = model(torch.cat([x, radial, coord], dim=1), t)

        a = alphas[t_step]
        a_hat = alphas_cumprod[t_step]

        if t_step > 0:
            noise = torch.randn_like(x)
        else:
            noise = 0

        x = (1/torch.sqrt(a)) * (x - ((1-a)/torch.sqrt(1-a_hat))*p_noise) + torch.sqrt(betas[t_step])*noise

        if t_step % step_interval == 0 or t_step == 0:

            img = (x.clamp(-1,1)+1)/2
            img = img[0,0].cpu().numpy()

            frames.append((img*255).astype("uint8"))

    gif_path = f"{gif_dir}/diffusion_process.gif"

    imageio.mimsave(
        gif_path,

```

```

        frames,
        duration=0.05
    )

    print("Saved GIF:", gif_path)

    return gif_path

```

```
gif_path = generate_single_diffusion_gif(ema_model, full_dataset)
```

```

Generating diffusion trajectory...
Saved GIF: /content/diffusion_single/diffusion_process.gif

```

```

from IPython.display import Image

Image(filename=gif_path)

```



```

from IPython.display import Image

Image(filename="/content/diffusion_gifs/sample_23.gif")

```



```

import shutil
from google.colab import files

folder_path = "/content/diffusion_gifs"
zip_path = "/content/diffusion_gifs.zip"

shutil.make_archive("/content/diffusion_gifs", 'zip', folder_path)

files.download(zip_path)

```

```

import shutil
from google.colab import files

folder_path = "/content/diffusion_single"
zip_path = "/content/diffusion_single.zip"

shutil.make_archive("/content/diffusion_single", 'zip', folder_path)

files.download(zip_path)

```

```

!pip install pytorch-fid -q

!python -m pytorch_fid "/content/real_fid" "/content/gen_fid" --device cuda

100% 20/20 [00:04<00:00, 4.29it/s]
100% 20/20 [00:04<00:00, 4.80it/s]
FID: 2.33970392349309

```

```

import os
import base64
from IPython.display import display, HTML

gif_dir = "/content/diffusion_gifs"
gifs = sorted([f for f in os.listdir(gif_dir) if f.endswith(".gif")])

html = '<div style="display:grid; grid-template-columns:repeat(5,1fr); gap:10px;">'

for g in gifs:
    path = os.path.join(gif_dir, g)

    with open(path, "rb") as f:
        data = base64.b64encode(f.read()).decode()

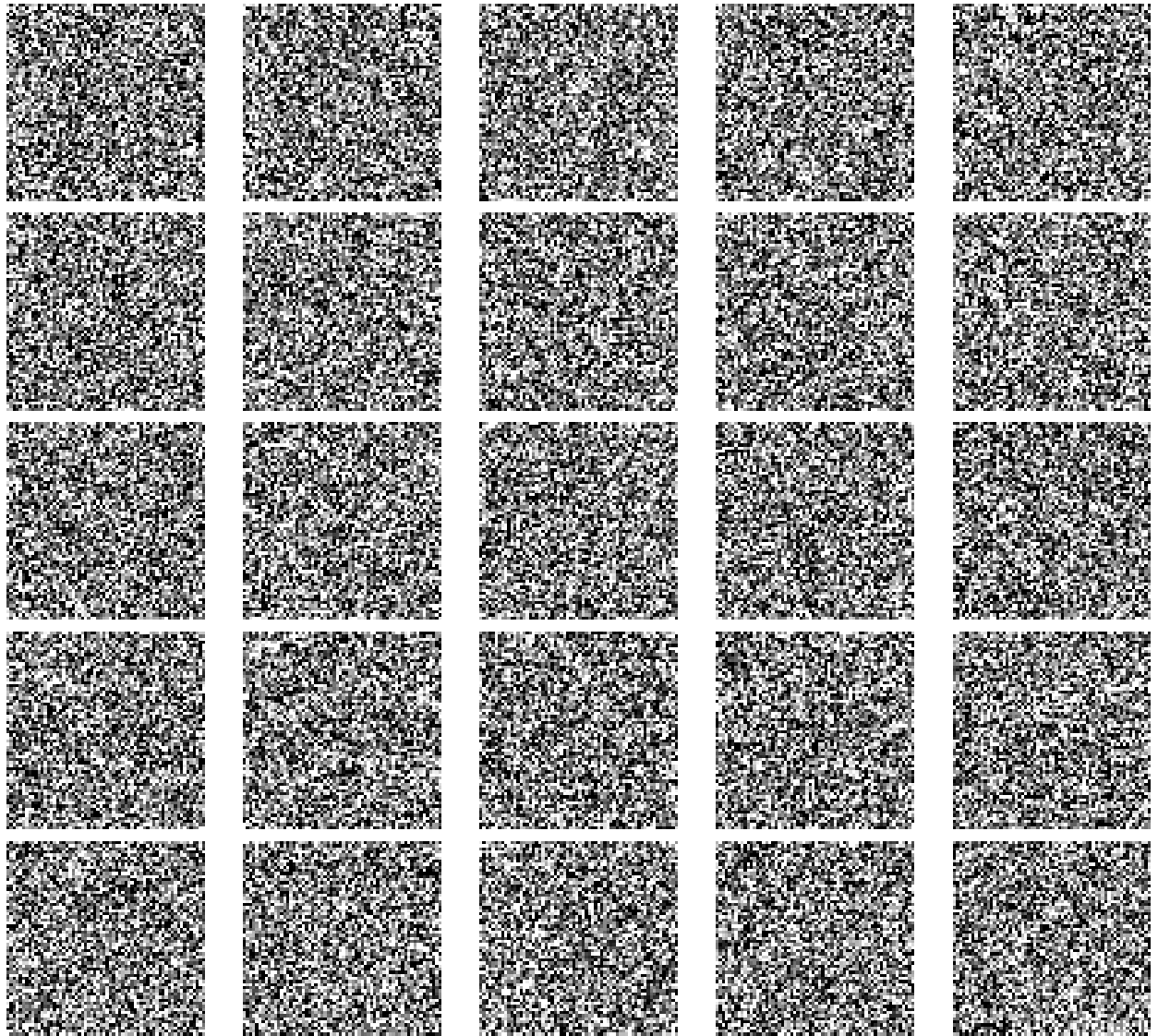
    html += f''

html += "</div>"

```

name = "system"

display(HTML(html))



Start coding or [generate](#) with AI.